In [191…

```
import pandas as pd
df=pd.read_csv("C:/Users/DELL/Downloads/city_day.csv")
```

In [192…

```
df.shape
```

Out[192…

```
(29531, 16)
```

In [193…

```
df.head()
```

Out[193…

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Ben
0	Ahmedabad	2015-01-01			0.92	18.22	17.15		0.92	27.64	133.36	
1	Ahmedabad	2015-01-02			0.97	15.69	16.46		0.97	24.55	34.06	
2	Ahmedabad	2015-01-03			17.4	19.3	29.7		17.4	29.07	30.7	
3	Ahmedabad	2015-01-04			1.7	18.48	17.97		1.7	18.59	36.08	
4	Ahmedabad	2015-01-05			22.1	21.42	37.76		22.1	39.33	39.31	

In [194…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
 #   Column              Non-Null Count  Dtype 
---  ------              --------------  ----- 
 0   City                29531 non-null  object
 1    Date               29531 non-null  object
 2    PM2.5              29531 non-null  object
 3    PM10               29531 non-null  object
 4    NO                 29531 non-null  object
 5    NO2                29531 non-null  object
 6    NOx                29531 non-null  object
 7    NH3                29531 non-null  object
 8    CO                 29531 non-null  object
 9    SO2                29531 non-null  object
 10   O3                 29531 non-null  object
 11   Benzene            29531 non-null  object
 12   Toluene            29531 non-null  object
 13   Xylene             29531 non-null  object
 14   AQI                29531 non-null  object
 15   AQI_Bucket         29531 non-null  object
dtypes: object(16)
memory usage: 3.6+ MB
```

In [195…

```
df.tail()
```

Out[195…

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O
29526	Visakhapatnam	2020-06-27	15.02	50.94	7.68	25.06	19.54	12.47	0.47	8.55	23.
29527	Visakhapatnam	2020-06-28	24.38	74.09	3.42	26.06	16.53	11.99	0.52	12.72	30.1
29528	Visakhapatnam	2020-06-29	22.91	65.73	3.45	29.53	18.33	10.71	0.48	8.42	30.9
29529	Visakhapatnam	2020-06-30	16.64	49.97	4.05	29.26	18.8	10.03	0.52	9.84	28.
29530	Visakhapatnam	2020-07-01	15.0	66.0	0.4	26.85	14.05	5.2	0.59	2.1	17.0

In [196…

```
df.describe()
```

Out[196…

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2
count	29531	29531	29531	29531	29531	29531	29531	29531	29531	29531
unique	26	2009	11717	12572	5777	7405	8157	5923	1780	4762
top	Ahmedabad	2020-06-26							0.0	
freq	2009	26	4598	11140	3582	3585	4185	10328	2328	3854

In [197…

```
print(df.isnull().sum())
```

```
City                 0
 Date                0
 PM2.5               0
 PM10                0
 NO                  0
 NO2                 0
 NOx                 0
 NH3                 0
 CO                  0
 SO2                 0
 O3                  0
 Benzene             0
 Toluene             0
 Xylene              0
 AQI                 0
 AQI_Bucket          0
dtype: int64
```

In [198…

```
print(df.dtypes)
```

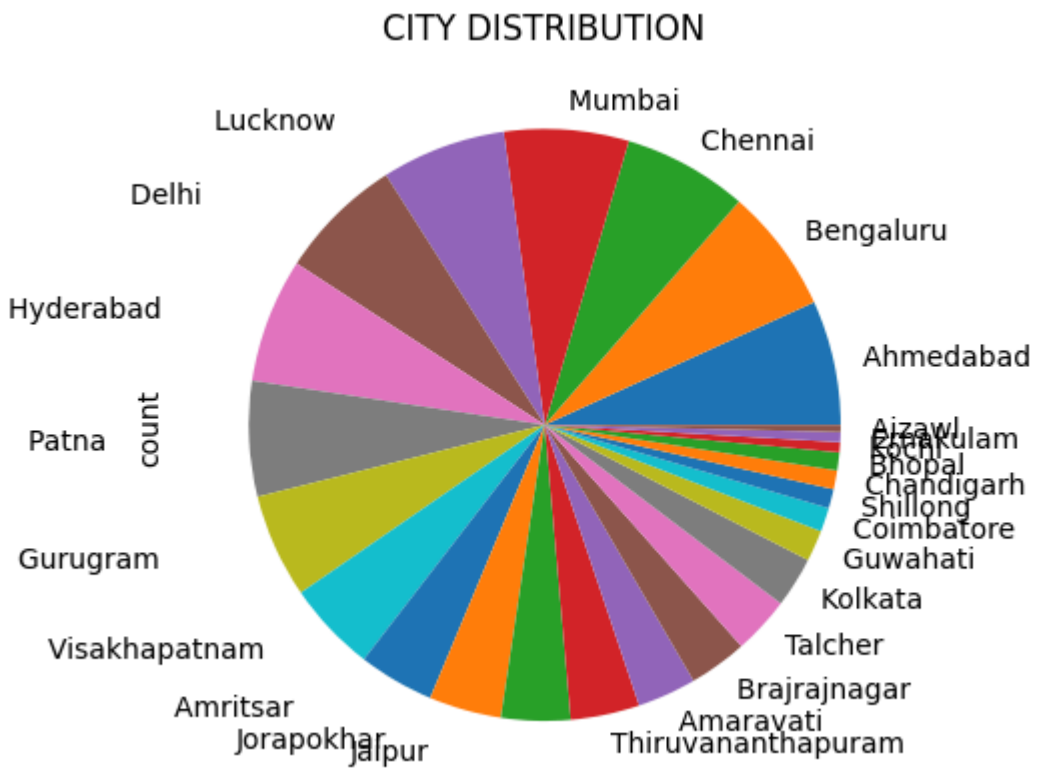
```
City                  object
 Date                 object
 PM2.5                object
 PM10                 object
 NO                   object
 NO2                  object
 NOx                  object
 NH3                  object
 CO                   object
 SO2                  object
 O3                   object
 Benzene              object
 Toluene              object
 Xylene               object
 AQI                  object
 AQI_Bucket           object
dtype: object
```

In [199…

```
df.columns=df.columns.str.strip()
```

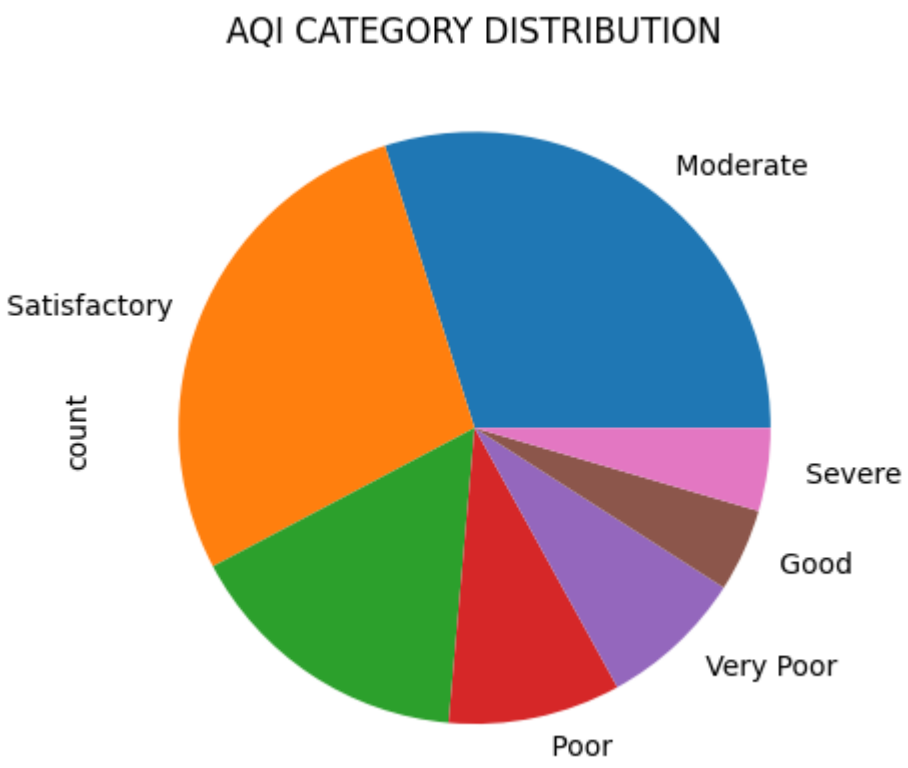
In [200…

```
import matplotlib.pyplot as plt
df["City"].value_counts().plot(kind='pie',color="pink")
plt.title("CITY DISTRIBUTION")
plt.show()
```



In [201…

```
df["AQI_Bucket"].value_counts().plot(kind='pie')
plt.title("AQI CATEGORY DISTRIBUTION")
plt.show()
```



In [202…
```
df["AQI_Bucket"].unique()
```

Out[202…
```
array([' ', ' Poor', ' Very Poor', ' Severe', ' Moderate',
       ' Satisfactory', ' Good'], dtype=object)
```

In [203…
```
df["AQI_Bucket"] = df["AQI_Bucket"].astype(str).str.strip()
```

In [205…
```
df["AQI_Bucket"] = df["AQI_Bucket"].astype(str).str.strip()
df = df[df["AQI_Bucket"] != '']
```

In [206…
```
df = df.dropna(subset=["AQI_Bucket"])
```

In [207…
```
df["AQI_Bucket"]=df["AQI_Bucket"].replace(['Poor','Very Poor','Severe'],'Bad')
```

In [208…
```
df["AQI_Bucket"].unique()
```

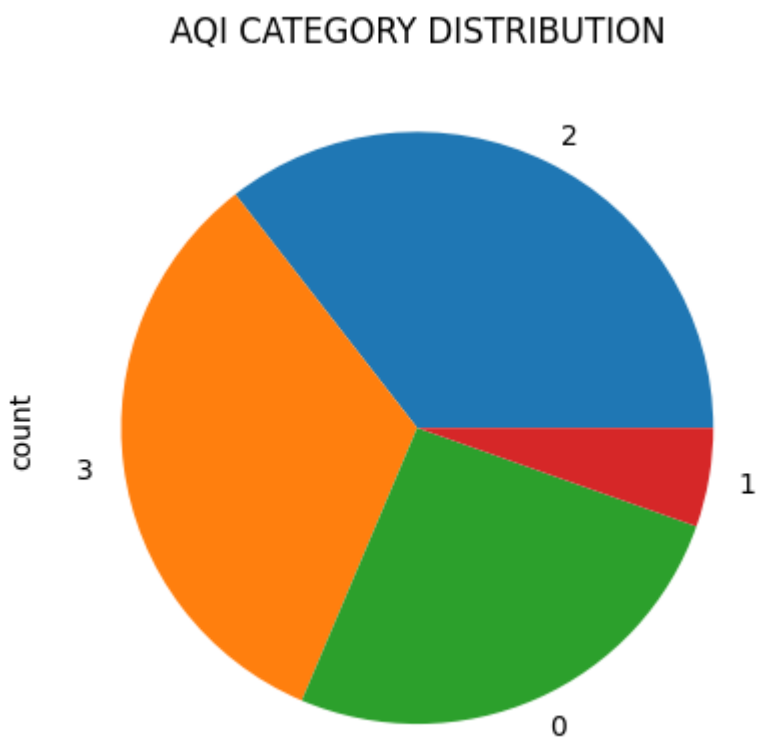
Out[208…
```
array(['Bad', 'Moderate', 'Satisfactory', 'Good'], dtype=object)
```

In [209…
```
from sklearn.preprocessing import LabelEncoder
le1 = LabelEncoder()
df['AQI_Bucket'] = le1.fit_transform(df['AQI_Bucket'])
```

In [210…
```
print(le1.classes_)
```

```
['Bad' 'Good' 'Moderate' 'Satisfactory']
```

In [211…
```
df["AQI_Bucket"].value_counts().plot(kind='pie')
plt.title("AQI CATEGORY DISTRIBUTION")
plt.show()
```



In [212…
```
df = df.apply(pd.to_numeric, errors='coerce')
```

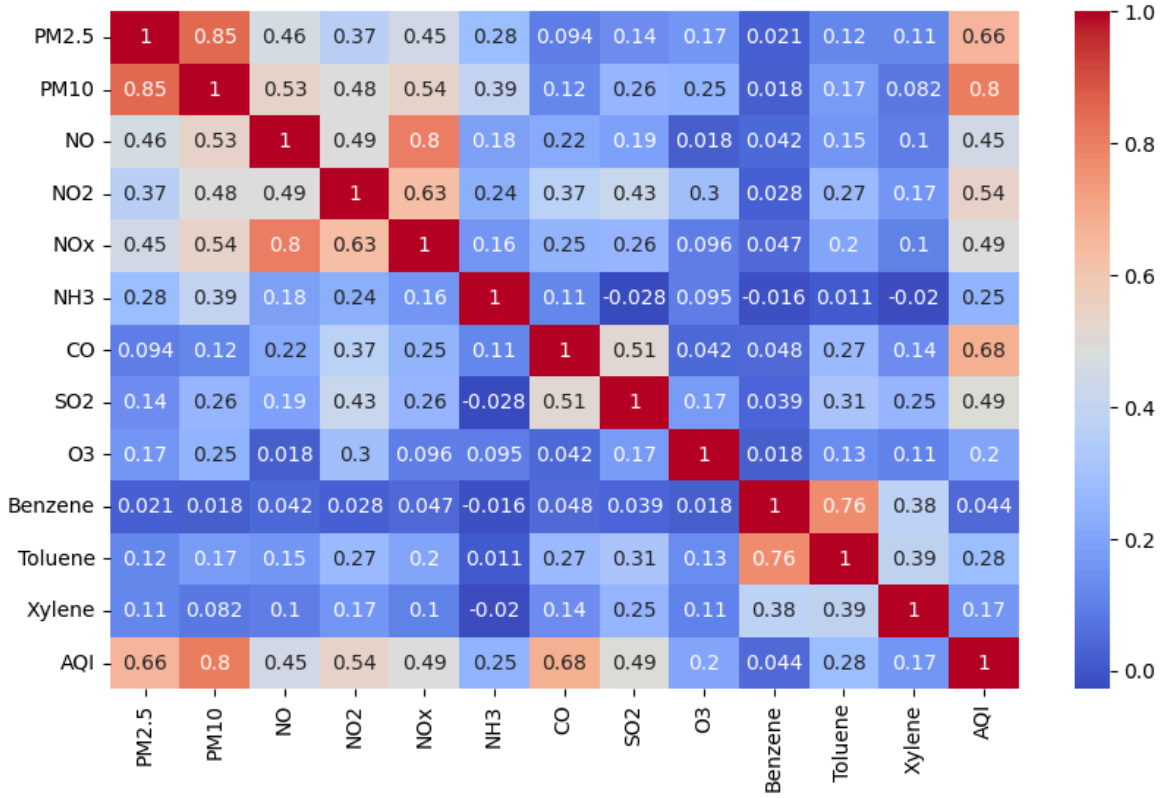
In [213…
```
print(df.dtypes)
```

```
City           float64
Date           float64
PM2.5          float64
PM10           float64
NO             float64
NO2            float64
NOx            float64
NH3            float64
CO             float64
SO2            float64
O3             float64
Benzene        float64
Toluene        float64
Xylene         float64
AQI            float64
AQI_Bucket       int64
dtype: object
```

In [216…
```
df=df.drop(["City","Date"],axis=1)
```

In [214…
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [217…
```
import seaborn as sns
import matplotlib.pyplot as plt
corr_df = df.drop(["AQI_Bucket"], axis=1, errors='ignore')
plt.figure(figsize=(10,6))
sns.heatmap(corr_df.corr(), annot=True, cmap='coolwarm')
plt.show()
```



In [25]:
```
#The correlation analysis shows that AQI is primarily influenced by particulate
#  and gases like CO and NO2, while features like Benzene and Xylene have neglig
#  High inter-feature correlation indicates multicollinearity, requiring feature
# or dimensionality reduction.
```

In [218…
```
df=df.drop_duplicates()
```

In [219…
```
df.shape
```

Out[219…
```
(24843, 14)
```

In [220…
```
df["AQI_Bucket"].value_counts()
```

Out[220…
```
AQI_Bucket
2    8827
3    8219
0    6456
1    1341
Name: count, dtype: int64
```

In [221…
```
import numpy as np
for col in df.columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(col, "Outliers:", len(outliers))
```

```
PM2.5 Outliers: 1911
PM10 Outliers: 1023
NO Outliers: 2342
NO2 Outliers: 1072
NOx Outliers: 1652
NH3 Outliers: 958
CO Outliers: 2289
SO2 Outliers: 2409
O3 Outliers: 664
Benzene Outliers: 1458
Toluene Outliers: 2123
Xylene Outliers: 859
AQI Outliers: 1358
AQI_Bucket Outliers: 0
```

In [222…

```
for col in df.columns:
    if col != 'AQI_Bucket':
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower) & (df[col] <= upper)]
```

In [223…

```
import numpy as np
for col in df.columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(col, "Outliers:", len(outliers))
```

```
PM2.5 Outliers: 15
PM10 Outliers: 34
NO Outliers: 128
NO2 Outliers: 13
NOx Outliers: 64
NH3 Outliers: 209
CO Outliers: 37
SO2 Outliers: 40
O3 Outliers: 52
Benzene Outliers: 36
Toluene Outliers: 32
Xylene Outliers: 35
AQI Outliers: 28
AQI_Bucket Outliers: 28
```

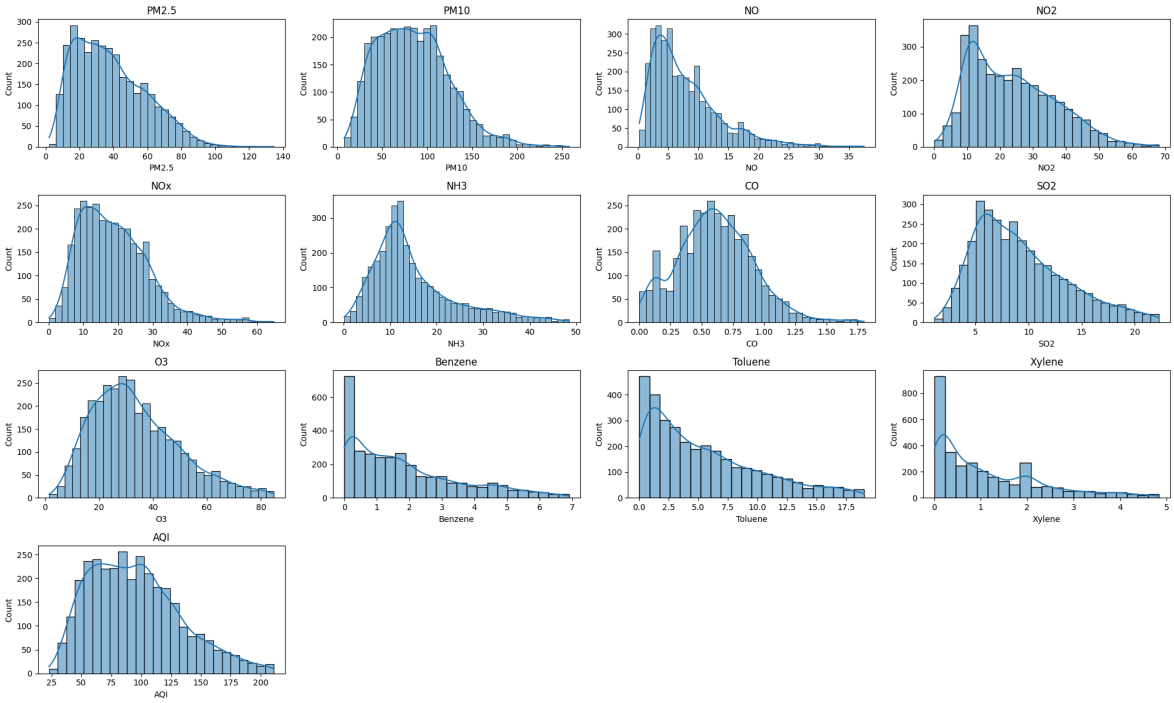
In [224…

```
for col in df.columns:
    skew = df[col].skew()
    print(col, "Skewness:", skew)
```

```
PM2.5 Skewness: 0.6884132272518679
PM10 Skewness: 0.6085904719161149
NO Skewness: 1.4810502337859086
NO2 Skewness: 0.6341756985065002
NOx Skewness: 0.9894409813731563
NH3 Skewness: 1.2738044998281357
CO Skewness: 0.3767558888095024
SO2 Skewness: 0.7826363286735283
O3 Skewness: 0.721786530863459
Benzene Skewness: 0.9713662232861626
Toluene Skewness: 0.9390992671343837
Xylene Skewness: 1.1574441093556866
AQI Skewness: 0.6218162971527902
AQI_Bucket Skewness: -0.7610146392238023
```

In [225…

```
import matplotlib.pyplot as plt
import seaborn as sns
cols = df.select_dtypes(include=['int64','float64']).columns
cols = cols.drop("AQI_Bucket", errors='ignore')
n_cols = 4
n_rows = (len(cols) // n_cols) + 1
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 12))
axes = axes.flatten()
for i, col in enumerate(cols):
    sns.histplot(df[col].dropna(), kde=True, ax=axes[i])
    axes[i].set_title(col)
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()
```



In [34]:

```
#Most features such as PM2.5, PM10, NO, NO2, NOx, CO, SO2, Benzene, Toluene, Xyl
# skewed (right-skewed), indicating a concentration of lower values with a long
#  O3 shows near-normal distribution, and no feature exhibits significant left s
```

In [226…

```
#for right skewed data
for col in df.columns:
```

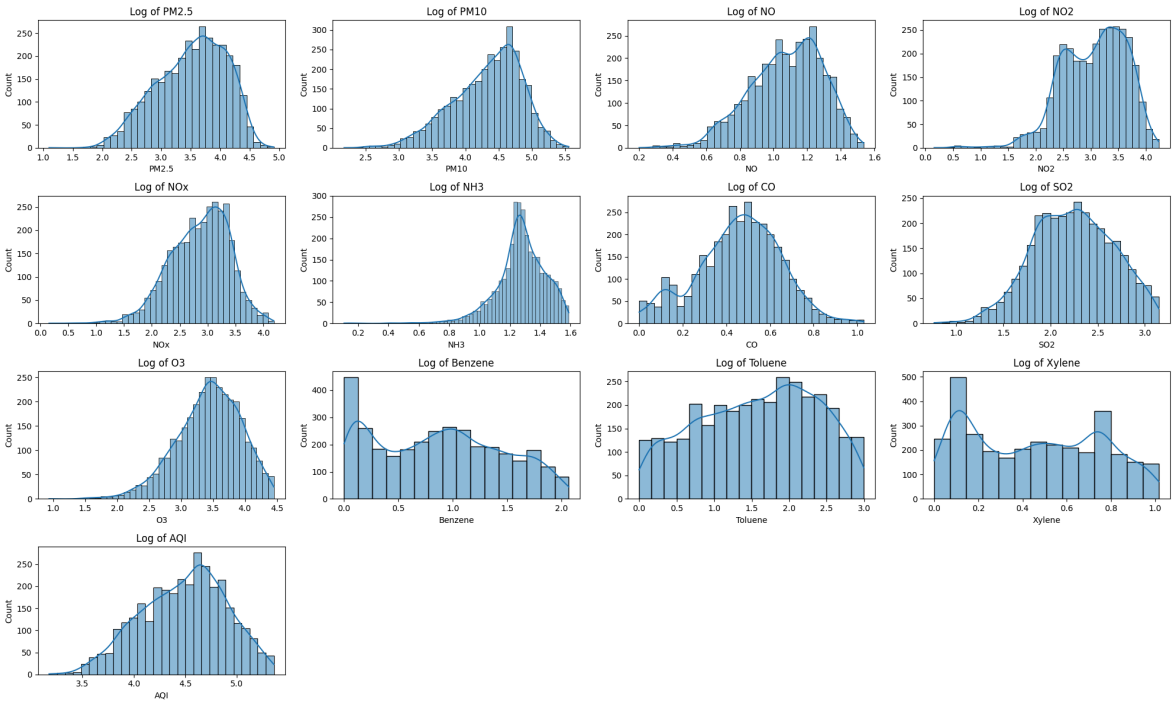
```
    if df[col].skew() > 1:
        df[col] = np.log1p(df[col])
```

In [227…

```
#for left skewed data
for col in df.columns:
    if df[col].skew() < -1:
        df[col] = np.square(df[col])
```

In [228…

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
cols = df.select_dtypes(include=['int64','float64']).columns
cols = cols.drop("AQI_Bucket", errors='ignore')
n_cols = 4
n_rows = (len(cols) // n_cols) + 1
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 12))
axes = axes.flatten()
for i, col in enumerate(cols):
    data = df[col]
    data = data[data >= 0]
    sns.histplot(np.log1p(data.dropna()), kde=True, ax=axes[i])
    axes[i].set_title("Log of " + col)
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()
```



In [229…

```
quantiles = df.quantile([0.25, 0.5, 0.75, 1.0])
print(quantiles.T)
```

```
                 0.25       0.50        0.75        1.00
PM2.5       21.095000  34.730000   51.920000  134.610000
PM10        53.585000  81.090000  109.340000  257.560000
NO           1.543298   1.987874    2.439299    3.649359
NO2         12.750000  21.880000   32.745000   68.180000
NOx         11.180000  17.470000   24.735000   64.810000
NH3          2.316981   2.590767    2.978332    3.899950
CO           0.400000   0.600000    0.810000    1.800000
SO2          5.880000   8.420000   11.875000   22.290000
O3          21.715000  31.160000   43.495000   84.770000
Benzene      0.390000   1.410000    2.810000    6.900000
Toluene      1.590000   4.150000    8.230000   19.000000
Xylene       0.173953   0.565314    1.078410    1.764731
AQI         65.000000  91.000000  119.000000  211.000000
AQI_Bucket   2.000000   2.000000    3.000000    3.000000
```

In [230…

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_mat
from sklearn.feature_selection import SequentialFeatureSelector
```

In [231…

```
le1.classes_
```

Out[231…

```
array(['Bad', 'Good', 'Moderate', 'Satisfactory'], dtype=object)
```

In [232…

```
X = df.drop('AQI_Bucket', axis=1)
y = df['AQI_Bucket']
```

In [233…

```
X
```

Out[233…

	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	To
2137	74.28	141.22	1.960095	44.97	28.88	2.817204	0.09	21.90	60.62	0.26	
2138	50.32	102.77	1.004302	33.85	19.41	2.607124	0.10	13.65	68.15	0.20	
2139	58.47	115.27	1.780024	41.64	26.15	2.785011	0.16	18.37	73.75	0.23	
2140	89.35	131.48	2.193886	42.10	28.88	3.101892	0.24	7.42	44.67	0.28	
2141	64.42	99.74	2.104134	34.78	24.36	2.924773	0.15	5.81	50.16	0.24	
...	...	...	...	...	...	...	...	...	...	...	
29525	7.63	32.27	1.932970	23.27	17.19	2.497329	0.46	6.87	19.90	1.45	
29526	15.02	50.94	2.161022	25.06	19.54	2.600465	0.47	8.55	23.30	2.24	
29527	24.38	74.09	1.486140	26.06	16.53	2.564180	0.52	12.72	30.14	0.74	
29528	22.91	65.73	1.492904	29.53	18.33	2.460443	0.48	8.42	30.96	0.01	
29529	16.64	49.97	1.619388	29.26	18.80	2.400619	0.52	9.84	28.30	0.00	

3275 rows × 13 columns

In [234…

```
y
```

Out[234…

```
2137     2
2138     2
2139     2
2140     2
2141     2
        ..
29525    1
29526    1
29527    3
29528    3
29529    3
Name: AQI_Bucket, Length: 3275, dtype: int64
```

In [235…

```
from sklearn.preprocessing import StandardScaler
SC=StandardScaler()
X_SC=SC.fit_transform(X)
```

In [236…

```
X_train, X_test, y_train, y_test = train_test_split(X_SC, y, test_size=0.2, rand
```

In [237…

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression(
    max_iter=3000,
    class_weight='balanced',
    C=10
)
```

In [238…

```
fs = SequentialFeatureSelector(model, n_features_to_select='auto', direction='fo
```

In [239…

```
fs.fit(X_SC,y)
```

Out[239…

```
▸ SequentialFeatureSelector
  ▸ estimator: LogisticRegression
    ▸ LogisticRegression
```

In [162…

```
selected_features = X.columns[fs.get_support()]
selected_features
```

Out[162…

```
Index(['NO', 'NO2', 'NOx', 'CO', 'Toluene', 'AQI'], dtype='object')
```

In [240…

```
be = SequentialFeatureSelector(model, n_features_to_select='auto', direction='ba
```

In [241…

```
be.fit(X_SC,y)
```

Out[241…

```
▸ SequentialFeatureSelector
  ▸ estimator: LogisticRegression
    ▸ LogisticRegression
```

In [242…

```
selected_features = X.columns[be.get_support()]
selected_features
```

Out[242…

```
Index(['PM2.5', 'NO', 'NO2', 'CO', 'Toluene', 'Xylene', 'AQI'], dtype='object')
```

In [243…

```
X_train, X_test, y_train, y_test = train_test_split(X_SC, y, test_size=0.3, rand
```

In [244…

```
X_SC.shape
```

Out[244…

```
(3275, 13)
```

In [245…

```
X_train, X_test, y_train, y_test
```

Out[245…

```
(array([[-0.55779416, -0.68614286,  0.29417253, ...,  0.63631164,
         -0.98135148, -0.97607679],
        [-0.81419306, -0.95653893,  0.50877712, ...,  1.78629904,
         -0.70609264, -0.3208319 ],
        [ 0.59600087,  0.99595759,  0.03807491, ..., -0.51367575,
          0.97415824,  1.04207745],
        ...,
        [ 0.2382908 , -0.05185869, -0.13767689, ..., -0.59933589,
         -1.08807986,  1.98563009],
        [-0.94959472, -0.64809085, -1.13271193, ..., -1.12400425,
          0.03544958, -0.81881802],
        [-0.38974245, -0.1015027 , -0.76026461, ...,  1.26805518,
          0.23611662, -0.3470417 ]], shape=(2292, 13)),
 array([[-0.8799733 , -1.12663897,  1.69006071, ..., -0.09394106,
         -1.06990052, -0.47809068],
        [-0.40462703, -0.20935873, -1.45365126, ...,  0.03454915,
          0.14824406,  0.7799795 ],
        [ 0.66322156,  1.51457373,  0.65504802, ..., -0.84560879,
         -0.67601616,  0.72755991],
        ...,
        [-0.99232787, -0.67631486,  0.17316262, ..., -0.08751655,
          1.43185516, -0.68776904],
        [ 0.70067308,  0.98990959,  1.62270429, ...,  0.99180123,
          1.6821872 ,  1.27796561],
        [-0.23849591, -0.1070467 , -0.99338099, ..., -0.58862837,
         -0.55994739, -0.13736334]], shape=(983, 13)),
 16514    3
 11898    3
 14057    2
 29411    2
 29399    3
         ..
 12176    2
 12226    2
 14960    2
 7779     3
 29421    3
 Name: AQI_Bucket, Length: 2292, dtype: int64,
 16501    3
 14852    2
 15400    2
 2722     3
 3052     3
         ..
 12128    2
 11950    2
 15520    3
 28335    2
 15786    3
 Name: AQI_Bucket, Length: 983, dtype: int64)
```

In [246…

```
model.fit(X_train, y_train)
```

Out[246…

```
▾ LogisticRegression
▶ Parameters
```

In [247…

```
df
```

Out[247…

	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	To
2137	74.28	141.22	1.960095	44.97	28.88	2.817204	0.09	21.90	60.62	0.26	
2138	50.32	102.77	1.004302	33.85	19.41	2.607124	0.10	13.65	68.15	0.20	
2139	58.47	115.27	1.780024	41.64	26.15	2.785011	0.16	18.37	73.75	0.23	
2140	89.35	131.48	2.193886	42.10	28.88	3.101892	0.24	7.42	44.67	0.28	
2141	64.42	99.74	2.104134	34.78	24.36	2.924773	0.15	5.81	50.16	0.24	
...	...	...	...	...	...	...	...	...	...	...	
29525	7.63	32.27	1.932970	23.27	17.19	2.497329	0.46	6.87	19.90	1.45	
29526	15.02	50.94	2.161022	25.06	19.54	2.600465	0.47	8.55	23.30	2.24	
29527	24.38	74.09	1.486140	26.06	16.53	2.564180	0.52	12.72	30.14	0.74	
29528	22.91	65.73	1.492904	29.53	18.33	2.460443	0.48	8.42	30.96	0.01	
29529	16.64	49.97	1.619388	29.26	18.80	2.400619	0.52	9.84	28.30	0.00	

3275 rows × 14 columns

In [248…

```
import numpy as np
sample = np.array([[ 4.38,74.09,3.42,26.06,16.53,11.99,0.52,12.72,30.14,0.74,2.2
sample_scaled = scaler.transform(sample)
pred = model.predict(sample_scaled)
print("Raw prediction:", pred)

result = le1.inverse_transform(pred)

print("Final AQI:", result[0])
```

```
Raw prediction: [0]
Final AQI: Bad
```

In [249…

```
print(cols)
```

```
Index(['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3',
       'Benzene', 'Toluene', 'Xylene', 'AQI'],
      dtype='object')
```

In [250…

```
print(le1.classes_)
```

```
['Bad' 'Good' 'Moderate' 'Satisfactory']
```

In [174…

```
import pickle
import pandas as pd
with open("logisticfinal.pkl", "wb") as f:
    pickle.dump((model, scaler, le1, cols), f)
```

In [251…

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
lr = LogisticRegression(max_iter=5000)
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'solver': ['lbfgs'],
}
```

```
grid_lr = GridSearchCV(
    lr,
    param_grid,
    cv=5,
    scoring='accuracy'
)
grid_lr.fit(X_train, y_train)
grid_lr.best_params_
best_model=grid_lr.best_estimator_
print(best_model.score(X_test,y_test))
print(grid_lr.best_params_)
print(grid_lr.best_estimator_)
```

```
0.9867751780264497
{'C': 10, 'solver': 'lbfgs'}
LogisticRegression(C=10, max_iter=5000)
```

In [252…

```
from sklearn.linear_model import LogisticRegression
model1=LogisticRegression(C=10,solver='lbfgs',max_iter=5000)
model1.fit(X_train,y_train)
```

Out[252…

```
▾ LogisticRegression
▸ Parameters
```

In [253…

```
y_pred = model1.predict(X_test)
```

In [254…

```
y_pred
```

Out[254…

```
array([3, 2, 2, 3, 3, 3, 1, 2, 3, 2, 3, 2, 2, 2, 2, 3, 3, 2, 3, 3, 3, 3,
       3, 3, 2, 2, 2, 2, 3, 3, 1, 0, 3, 2, 3, 3, 3, 3, 2, 2, 2, 2, 2, 3,
       0, 2, 2, 3, 3, 3, 3, 1, 2, 3, 1, 2, 3, 2, 3, 1, 3, 3, 2, 2, 3, 2,
       3, 2, 3, 3, 1, 3, 3, 2, 3, 1, 2, 2, 3, 2, 2, 2, 3, 1, 2, 2, 3, 1,
       3, 1, 3, 2, 3, 2, 2, 3, 3, 3, 1, 2, 1, 2, 3, 3, 2, 3, 2, 3, 3, 3,
       1, 3, 2, 3, 3, 3, 3, 3, 3, 2, 2, 3, 1, 3, 1, 3, 1, 3, 2, 2, 3, 3,
       3, 3, 2, 3, 2, 2, 3, 3, 2, 3, 2, 1, 2, 1, 3, 2, 3, 2, 3, 1, 3, 3,
       3, 3, 2, 3, 3, 3, 3, 2, 2, 2, 2, 3, 2, 3, 2, 3, 3, 3, 2, 2, 2, 3,
       3, 3, 1, 3, 1, 3, 2, 2, 2, 2, 3, 3, 3, 2, 1, 3, 2, 2, 2, 1, 2, 2,
       3, 3, 3, 3, 3, 3, 2, 2, 2, 3, 3, 1, 3, 0, 3, 3, 3, 3, 2, 3, 3, 1,
       2, 1, 3, 3, 2, 2, 3, 3, 2, 3, 2, 2, 3, 2, 2, 3, 3, 3, 2, 3, 2, 3,
       3, 1, 2, 2, 3, 3, 2, 3, 3, 2, 1, 1, 2, 1, 3, 3, 2, 1, 1, 1, 3, 1,
       2, 2, 2, 3, 3, 2, 3, 3, 3, 2, 2, 3, 2, 3, 3, 3, 1, 3, 2, 1, 2, 2,
       3, 2, 3, 2, 2, 3, 3, 3, 2, 3, 3, 3, 1, 2, 1, 3, 1, 3, 2, 3, 2, 2,
       3, 2, 3, 2, 1, 3, 3, 3, 3, 2, 1, 3, 2, 3, 2, 1, 3, 2, 2, 1, 1, 2,
       3, 1, 2, 2, 3, 3, 2, 2, 2, 2, 1, 3, 2, 2, 2, 3, 2, 3, 2, 2, 2, 2,
       3, 2, 2, 3, 3, 2, 2, 3, 2, 3, 2, 3, 2, 2, 2, 1, 3, 2, 3, 1, 2, 3,
       2, 2, 3, 1, 2, 2, 2, 3, 3, 2, 1, 3, 3, 2, 2, 2, 2, 3, 2, 2, 1, 2,
       3, 3, 3, 3, 3, 3, 3, 1, 3, 3, 2, 3, 3, 2, 2, 1, 3, 3, 3, 3, 3, 2,
       1, 2, 3, 2, 3, 1, 3, 3, 1, 2, 3, 3, 3, 3, 2, 2, 3, 3, 2, 2, 2, 1,
       2, 3, 3, 2, 3, 3, 2, 3, 3, 1, 3, 3, 3, 2, 3, 2, 3, 2, 2, 3, 2, 2,
       0, 2, 3, 1, 2, 3, 3, 3, 2, 3, 1, 3, 3, 2, 3, 3, 2, 2, 3, 1, 1, 3,
       2, 2, 3, 3, 3, 2, 3, 1, 3, 2, 2, 3, 3, 3, 2, 2, 3, 3, 2, 2, 2, 3,
       3, 2, 3, 3, 2, 2, 3, 3, 2, 3, 3, 2, 1, 3, 2, 2, 2, 3, 3, 2, 3, 3,
       3, 3, 2, 3, 2, 2, 3, 3, 3, 3, 3, 3, 2, 2, 3, 2, 2, 3, 3, 2, 3, 3,
       2, 3, 1, 2, 2, 2, 3, 3, 3, 2, 3, 3, 3, 3, 3, 2, 3, 2, 1, 3, 2, 3,
       2, 2, 2, 2, 3, 1, 3, 1, 3, 2, 2, 3, 1, 3, 3, 2, 3, 3, 2, 2, 3, 3,
       3, 2, 3, 2, 2, 2, 3, 2, 3, 3, 3, 1, 3, 3, 3, 3, 3, 1, 3, 2, 2, 2,
       3, 3, 1, 3, 2, 2, 1, 3, 2, 2, 2, 3, 3, 2, 2, 2, 2, 3, 2, 2, 3, 3,
       1, 3, 3, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 3, 2, 2, 3,
       3, 1, 3, 3, 3, 2, 3, 3, 3, 3, 3, 3, 3, 3, 1, 3, 3, 2, 3, 2, 3, 2,
       3, 2, 3, 2, 2, 2, 1, 1, 3, 2, 3, 3, 2, 2, 3, 3, 2, 3, 3, 3, 3, 2,
       3, 3, 3, 2, 2, 2, 3, 3, 2, 2, 3, 1, 2, 2, 2, 2, 1, 3, 2, 3, 3, 2,
       3, 3, 1, 1, 2, 3, 2, 3, 2, 3, 3, 2, 2, 3, 3, 2, 2, 2, 3, 2, 2, 1,
       3, 2, 2, 1, 3, 3, 1, 2, 3, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 3, 2, 3, 2, 3, 2, 3, 1, 3, 3, 2, 3, 3, 3, 2, 1, 1, 2, 2, 3, 2,
       3, 3, 3, 2, 3, 2, 3, 1, 3, 3, 2, 2, 3, 2, 1, 2, 2, 0, 3, 2, 3, 2,
       2, 2, 2, 3, 3, 2, 3, 3, 3, 2, 3, 3, 1, 3, 3, 3, 2, 2, 3, 3, 3, 2,
       3, 3, 2, 3, 2, 2, 2, 3, 3, 1, 2, 3, 2, 2, 2, 3, 2, 2, 3, 3, 2, 3,
       1, 3, 2, 1, 2, 3, 3, 2, 3, 2, 2, 3, 2, 3, 2, 3, 2, 3, 3, 2, 2, 3,
       1, 2, 3, 2, 3, 3, 2, 1, 2, 2, 3, 3, 1, 3, 3, 3, 3, 3, 2, 3, 2, 3,
       1, 3, 2, 2, 3, 3, 3, 3, 1, 3, 3, 3, 2, 2, 3, 3, 3, 2, 3, 3, 3, 3,
       3, 1, 1, 2, 3, 3, 2, 3, 2, 3, 3, 2, 2, 2, 2, 3, 3, 3, 1, 1, 3, 3,
       2, 2, 3, 2, 3, 3, 2, 2, 2, 2, 3, 2, 2, 3, 3, 3, 3, 3, 2, 3, 2, 2,
       1, 3, 3, 3, 1, 3, 2, 3, 3, 3, 2, 2, 3, 2, 3])
```

In [255…

```
y_test
```

Out[255…

```
16501    3
14852    2
15400    2
2722     3
3052     3
        ..
12128    2
11950    2
15520    3
28335    2
15786    3
Name: AQI_Bucket, Length: 983, dtype: int64
```

In [256…

```
slope = model.coef_[0]
intercept = model.intercept_
slope, intercept
```

Out[256…

```
(array([ 0.86913804,  1.13309075, -1.33620102, -1.11967441,  1.73770416,
         0.21117712, -0.05305868, -0.178539  ,  0.54650808,  0.33004092,
         0.47745158,  0.11730897, 21.51066201]),
 array([-20.33314094,  -9.7574302 ,  13.98734293,  16.10322821]))
```

In [257…

```
accuracy_score(y_test, y_pred)
```

Out[257…

```
0.9867751780264497
```

In [258…

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.83      0.45      0.59        11
           1       1.00      0.99      1.00       106
           2       0.98      0.99      0.98       387
           3       0.99      1.00      0.99       479

    accuracy                           0.99       983
   macro avg       0.95      0.86      0.89       983
weighted avg       0.99      0.99      0.99       983
```

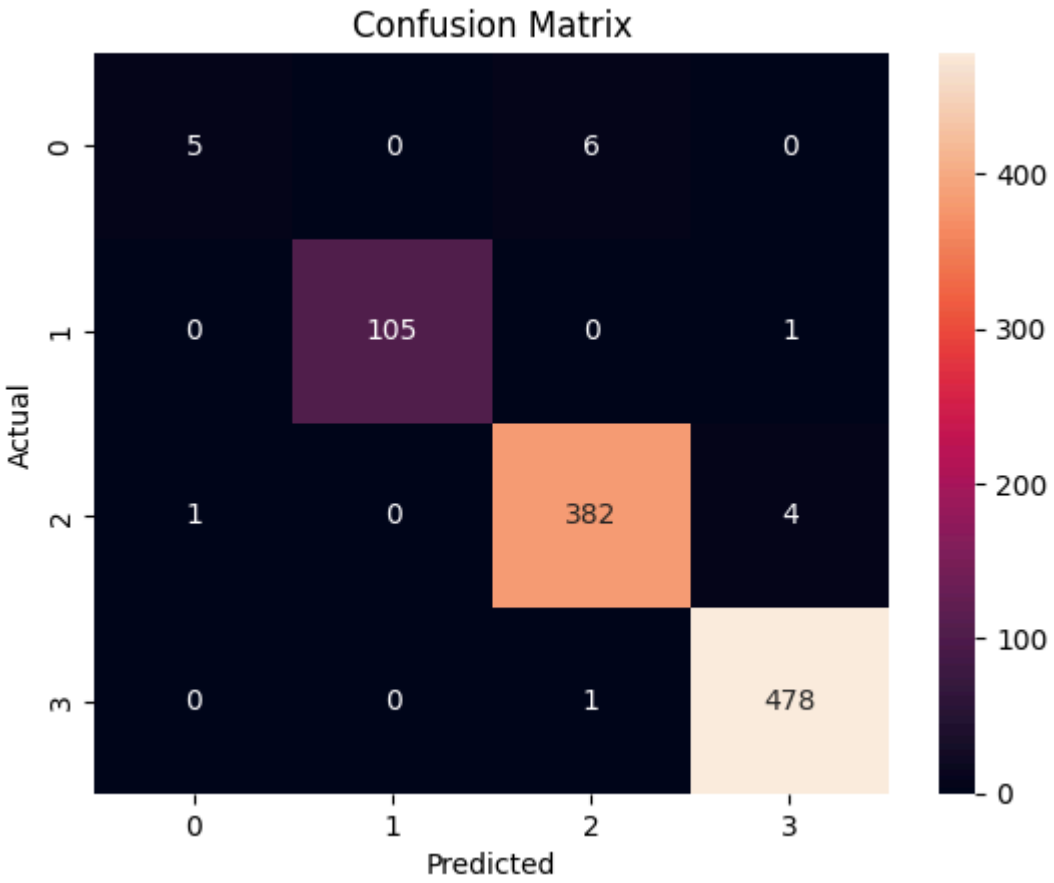
In [259…

```
print(confusion_matrix(y_test,y_pred))
```

```
[[  5   0   6   0]
 [  0 105   0   1]
 [  1   0 382   4]
 [  0   0   1 478]]
```

In [260…

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



In [261…

```
import pickle
import pandas as pd
with open("logfinal.pkl", "wb") as f:
    pickle.dump((model1, scaler, le1, cols), f)
```

In [262…

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.feature_selection import SequentialFeatureSelector
```

In [263…

```
kn =KNeighborsClassifier(n_neighbors=5)
```

In [264…

```
fs1=SequentialFeatureSelector(kn ,n_features_to_select='auto',direction='forward
```

In [265…

```
fs1.fit(X_SC,y)
```

Out[265…

```
▸ SequentialFeatureSelector
▸ estimator: KNeighborsClassifier
  ▸ KNeighborsClassifier
```

In [266…

```
y
```

Out[266…

```
2137     2
2138     2
2139     2
2140     2
2141     2
        ..
29525    1
29526    1
29527    3
29528    3
29529    3
Name: AQI_Bucket, Length: 3275, dtype: int64
```

In [267…

```
y_pred
```

Out[267…

```
array([3, 2, 2, 3, 3, 3, 1, 2, 3, 2, 3, 2, 2, 2, 2, 3, 3, 2, 3, 3, 3, 3,
       3, 3, 2, 2, 2, 2, 3, 3, 1, 0, 3, 2, 3, 3, 3, 3, 2, 2, 2, 2, 2, 3,
       0, 2, 2, 3, 3, 3, 3, 1, 2, 3, 1, 2, 3, 2, 3, 1, 3, 3, 2, 2, 3, 2,
       3, 2, 3, 3, 1, 3, 3, 2, 3, 1, 2, 2, 3, 2, 2, 2, 3, 1, 2, 2, 3, 1,
       3, 1, 3, 2, 3, 2, 2, 3, 3, 3, 1, 2, 1, 2, 3, 3, 2, 3, 2, 3, 3, 3,
       1, 3, 2, 3, 3, 3, 3, 3, 3, 2, 2, 3, 1, 3, 1, 3, 1, 3, 2, 2, 3, 3,
       3, 3, 2, 3, 2, 2, 3, 3, 2, 3, 2, 1, 2, 1, 3, 2, 3, 2, 3, 1, 3, 3,
       3, 3, 2, 3, 3, 3, 3, 2, 2, 2, 2, 3, 2, 3, 2, 3, 3, 3, 2, 2, 2, 3,
       3, 3, 1, 3, 1, 3, 2, 2, 2, 2, 3, 3, 3, 2, 1, 3, 2, 2, 2, 1, 2, 2,
       3, 3, 3, 3, 3, 3, 2, 2, 2, 3, 3, 1, 3, 0, 3, 3, 3, 3, 2, 3, 3, 1,
       2, 1, 3, 3, 2, 2, 3, 3, 2, 3, 2, 2, 3, 2, 2, 3, 3, 3, 2, 3, 2, 3,
       3, 1, 2, 2, 3, 3, 2, 3, 3, 2, 1, 1, 2, 1, 3, 3, 2, 1, 1, 1, 3, 1,
       2, 2, 2, 3, 3, 2, 3, 3, 3, 2, 2, 3, 2, 3, 3, 3, 1, 3, 2, 1, 2, 2,
       3, 2, 3, 2, 2, 3, 3, 3, 2, 3, 3, 3, 1, 2, 1, 3, 1, 3, 2, 3, 2, 2,
       3, 2, 3, 2, 1, 3, 3, 3, 3, 2, 1, 3, 2, 3, 2, 1, 3, 2, 2, 1, 1, 2,
       3, 1, 2, 2, 3, 3, 2, 2, 2, 2, 1, 3, 2, 2, 2, 3, 2, 3, 2, 2, 2, 2,
       3, 2, 2, 3, 3, 2, 2, 3, 2, 3, 2, 3, 2, 2, 2, 1, 3, 2, 3, 1, 2, 3,
       2, 2, 3, 1, 2, 2, 2, 3, 3, 2, 1, 3, 3, 2, 2, 2, 2, 3, 2, 2, 1, 2,
       3, 3, 3, 3, 3, 3, 3, 1, 3, 3, 2, 3, 3, 2, 2, 1, 3, 3, 3, 3, 3, 2,
       1, 2, 3, 2, 3, 1, 3, 3, 1, 2, 3, 3, 3, 3, 2, 2, 3, 3, 2, 2, 2, 1,
       2, 3, 3, 2, 3, 3, 2, 3, 3, 1, 3, 3, 3, 2, 3, 2, 3, 2, 2, 3, 2, 2,
       0, 2, 3, 1, 2, 3, 3, 3, 2, 3, 1, 3, 3, 2, 3, 3, 2, 2, 3, 1, 1, 3,
       2, 2, 3, 3, 3, 2, 3, 1, 3, 2, 2, 3, 3, 3, 2, 2, 3, 3, 2, 2, 2, 3,
       3, 2, 3, 3, 2, 2, 3, 3, 2, 3, 3, 2, 1, 3, 2, 2, 2, 3, 3, 2, 3, 3,
       3, 3, 2, 3, 2, 2, 3, 3, 3, 3, 3, 3, 2, 2, 3, 2, 2, 3, 3, 2, 3, 3,
       2, 3, 1, 2, 2, 2, 3, 3, 3, 2, 3, 3, 3, 3, 3, 2, 3, 2, 1, 3, 2, 3,
       2, 2, 2, 2, 3, 1, 3, 1, 3, 2, 2, 3, 1, 3, 3, 2, 3, 3, 2, 2, 3, 3,
       3, 2, 3, 2, 2, 2, 3, 2, 3, 3, 3, 1, 3, 3, 3, 3, 3, 1, 3, 2, 2, 2,
       3, 3, 1, 3, 2, 2, 1, 3, 2, 2, 2, 3, 3, 2, 2, 2, 2, 3, 2, 2, 3, 3,
       1, 3, 3, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 3, 2, 2, 3,
       3, 1, 3, 3, 3, 2, 3, 3, 3, 3, 3, 3, 3, 3, 1, 3, 3, 2, 3, 2, 3, 2,
       3, 2, 3, 2, 2, 2, 1, 1, 3, 2, 3, 3, 2, 2, 3, 3, 2, 3, 3, 3, 3, 2,
       3, 3, 3, 2, 2, 2, 3, 3, 2, 2, 3, 1, 2, 2, 2, 2, 1, 3, 2, 3, 3, 2,
       3, 3, 1, 1, 2, 3, 2, 3, 2, 3, 3, 2, 2, 3, 3, 2, 2, 2, 3, 2, 2, 1,
       3, 2, 2, 1, 3, 3, 1, 2, 3, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 3, 2, 3, 2, 3, 2, 3, 1, 3, 3, 2, 3, 3, 3, 2, 1, 1, 2, 2, 3, 2,
       3, 3, 3, 2, 3, 2, 3, 1, 3, 3, 2, 2, 3, 2, 1, 2, 2, 0, 3, 2, 3, 2,
       2, 2, 2, 3, 3, 2, 3, 3, 3, 2, 3, 3, 1, 3, 3, 3, 2, 2, 3, 3, 3, 2,
       3, 3, 2, 3, 2, 2, 2, 3, 3, 1, 2, 3, 2, 2, 2, 3, 2, 2, 3, 3, 2, 3,
       1, 3, 2, 1, 2, 3, 3, 2, 3, 2, 2, 3, 2, 3, 2, 3, 2, 3, 3, 2, 2, 3,
       1, 2, 3, 2, 3, 3, 2, 1, 2, 2, 3, 3, 1, 3, 3, 3, 3, 3, 2, 3, 2, 3,
       1, 3, 2, 2, 3, 3, 3, 3, 1, 3, 3, 3, 2, 2, 3, 3, 3, 2, 3, 3, 3, 3,
       3, 1, 1, 2, 3, 3, 2, 3, 2, 3, 3, 2, 2, 2, 2, 3, 3, 3, 1, 1, 3, 3,
       2, 2, 3, 2, 3, 3, 2, 2, 2, 2, 3, 2, 2, 3, 3, 3, 3, 3, 2, 3, 2, 2,
       1, 3, 3, 3, 1, 3, 2, 3, 3, 3, 2, 2, 3, 2, 3])
```

In [268…

```
selected_features = X.columns[fs1.get_support()]
selected_features
```

Out[268…

```
Index(['PM2.5', 'PM10', 'NO', 'NOx', 'O3', 'AQI'], dtype='object')
```

In [269…

```
kn.fit(X_train,y_train)
```

Out[269…

```
▾ KNeighborsClassifier
▸ Parameters
```

In [270…
```
y_pred_kn=kn.predict(X_test)
```

In [271…
```
accuracy_score(y_test,y_pred_kn)
```

Out[271…
```
0.8229908443540183
```

In [272…
```
f1_score(y_test,y_pred_kn,average='weighted')
```

Out[272…
```
0.81988179005309
```

In [273…
```
print(confusion_matrix(y_test,y_pred_kn))
```

```
[[  1   0  10   0]
 [  0  79   0  27]
 [  1   0 319  67]
 [  0  26  43 410]]
```

In [274…
```
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier()
param_grid = {
    'n_neighbors': [3,5,7,9],
    'weights': ['uniform','distance'],
    'metric': ['euclidean','manhattan']
}
grid_knn = GridSearchCV(kn, param_grid, cv=5, scoring='accuracy')
grid_knn.fit(X_train, y_train)

print("Best KNN Parameters:", grid_knn.best_params_)
print("Best Accuracy:", grid_knn.best_score_)
```

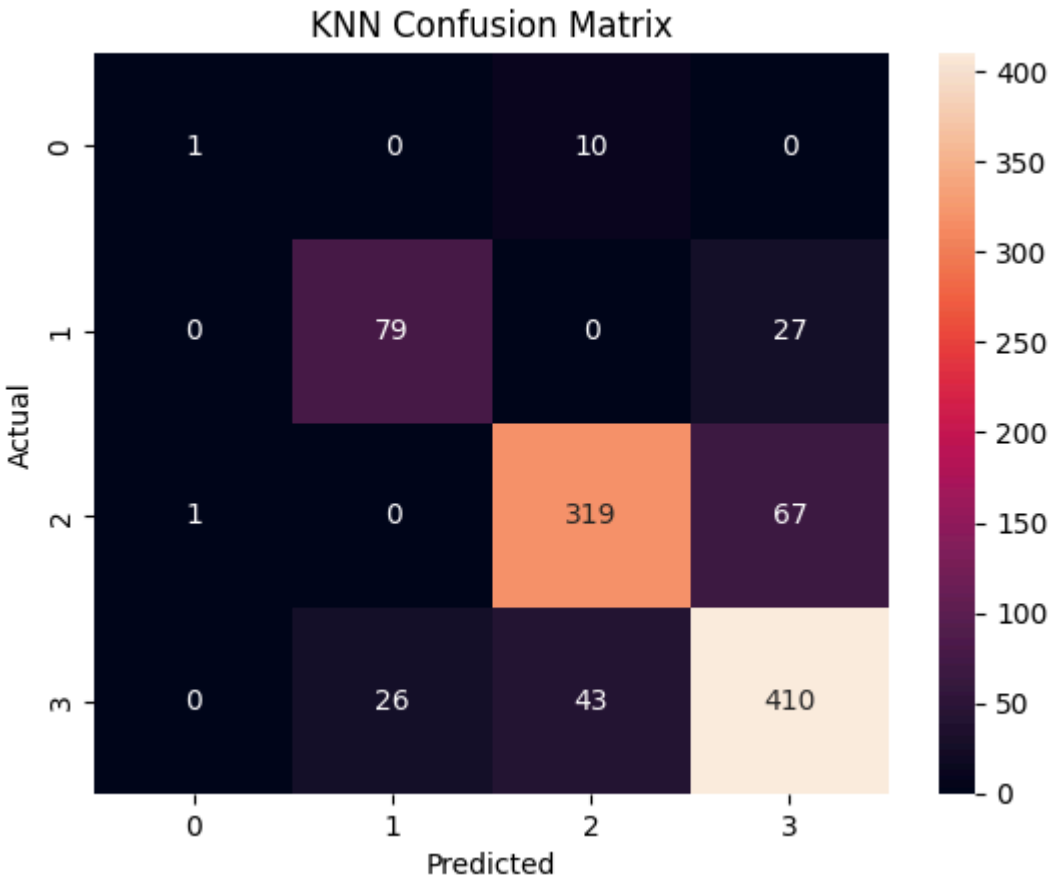
```
Best KNN Parameters: {'metric': 'manhattan', 'n_neighbors': 9, 'weights': 'distan
ce'}
Best Accuracy: 0.8638838941690213
```

In [275…
```
knn1=KNeighborsClassifier(n_neighbors=9,weights='distance',metric='manhattan')
knn1.fit(X_train,y_train)
```

Out[275…
```
▾ KNeighborsClassifier
▸ Parameters
```

In [277…
```
import pickle
import pandas as pd
with open("knnfinal.pkl", "wb") as f:
    pickle.dump((knn1, scaler, le1, cols), f)
```

In [276…
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_kn)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN Confusion Matrix")
plt.show()
```



In [278…

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(
    max_depth=4,
    min_samples_split=10,
    min_samples_leaf=5
)
dt.fit(X_train,y_train)
```

Out[278…

```
▾ DecisionTreeClassifier
▸ Parameters
```

In [279…

```
y_pred_dt = dt.predict(X_test)
```

In [280…

```
accuracy_score(y_test, y_pred_dt)
```

Out[280…

```
1.0
```

In [281…

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
dt = DecisionTreeClassifier(random_state=42)
param_grid = {
    'max_depth': [3, 5, 10, 15, None],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 5, 10],
    'criterion': ['gini', 'entropy']
}
grid_dt = GridSearchCV(
    dt,
    param_grid,
```

```
    cv=5,
    scoring='accuracy'
)
grid_dt.fit(X_train, y_train)
print("Best Parameters:", grid_dt.best_params_)
print("Best Accuracy:", grid_dt.best_score_)
```

```
Best Parameters: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'mi
n_samples_split': 2}
Best Accuracy: 0.9995633187772925
```

In [282…

```
dt1=DecisionTreeClassifier(criterion= 'gini', max_depth= 3, min_samples_leaf= 1,
dt1.fit(X_train,y_train)
```

Out[282…

```
▾ DecisionTreeClassifier
▸ Parameters
```

In [283…

```
import pickle
import pandas as pd
with open("dtfinal.pkl", "wb") as f:
    pickle.dump((dt1, scaler, le1, cols), f)
```

In [284…

```
print(classification_report(y_test,y_pred_kn))
```

```
              precision    recall  f1-score   support

           0       0.50      0.09      0.15        11
           1       0.75      0.75      0.75       106
           2       0.86      0.82      0.84       387
           3       0.81      0.86      0.83       479

    accuracy                           0.82       983
   macro avg       0.73      0.63      0.64       983
weighted avg       0.82      0.82      0.82       983
```

In [285…

```
print(confusion_matrix(y_test,y_pred_dt))
```

```
[[ 11   0   0   0]
 [  0 106   0   0]
 [  0   0 387   0]
 [  0   0   0 479]]
```

In [286…

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc
print(classification_report(y_test, y_pred_dt))
```

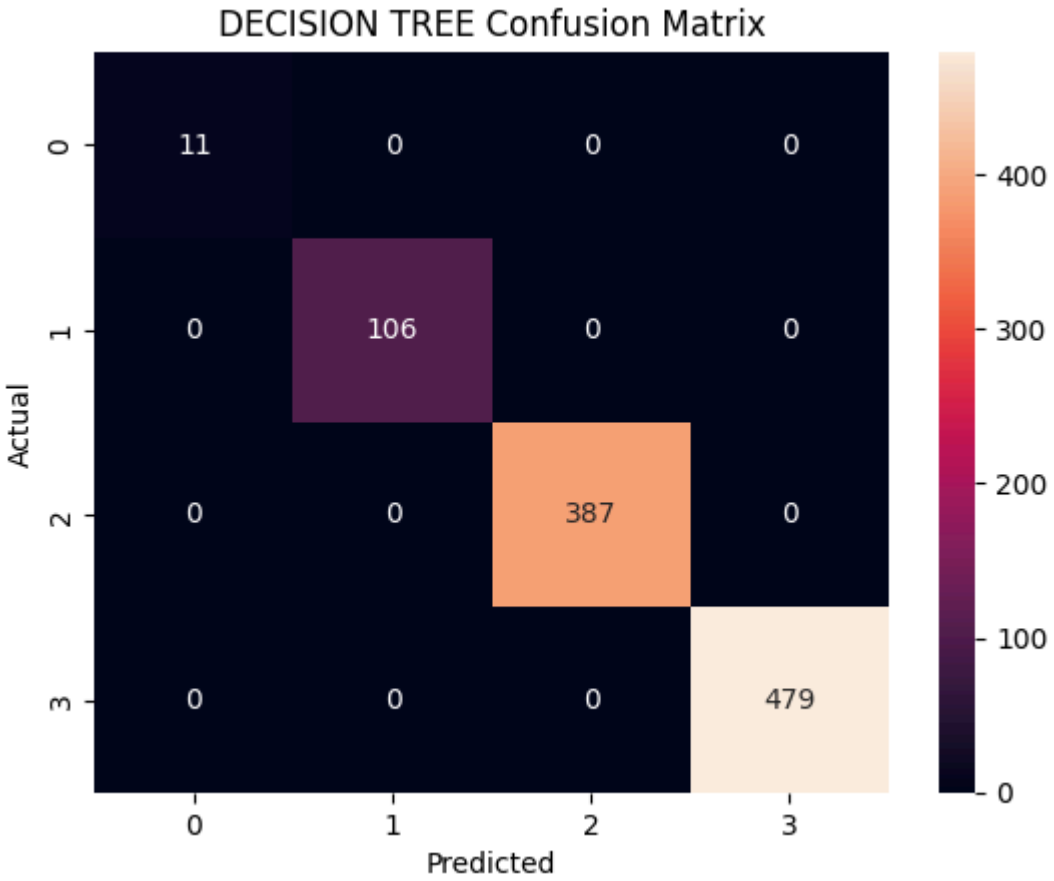
```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        11
           1       1.00      1.00      1.00       106
           2       1.00      1.00      1.00       387
           3       1.00      1.00      1.00       479

    accuracy                           1.00       983
   macro avg       1.00      1.00      1.00       983
weighted avg       1.00      1.00      1.00       983
```

In [287…

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_dt)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("DECISION TREE Confusion Matrix")
plt.show()
```



In [291…

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
```

Out[291…

```
▾ RandomForestClassifier
▸ Parameters
```

In [292…

```
y_pred_rf = rf.predict(X_test)
```

In [293…

```
accuracy_score(y_test, y_pred_rf)
```

Out[293…

```
0.9949135300101729
```

In [294…

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
rf = RandomForestClassifier(random_state=42)
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10],
```

```
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}
grid_rf = GridSearchCV(
    rf,
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)
grid_rf.fit(X_train, y_train)
print("Best Parameters:", grid_rf.best_params_)
print("Best Accuracy:", grid_rf.best_score_)
```

```
Best Parameters: {'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 1,
'min_samples_split': 10, 'n_estimators': 200}
Best Accuracy: 0.9982561292348089
```

In [297…

```
from sklearn.ensemble import RandomForestClassifier
rf1 = RandomForestClassifier(max_depth=10, max_features='sqrt', min_samples_leaf
rf1.fit(X_train, y_train)
```

Out[297…

```
▾ RandomForestClassifier
▸ Parameters
```

In [299…

```
import pickle
import pandas as pd
with open("rffinal.pkl", "wb") as f:
    pickle.dump((rf1, scaler, le1, cols), f)
```

In [300…

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc
print(classification_report(y_test, y_pred_rf))
```

```
              precision    recall  f1-score   support

           0       1.00      0.55      0.71        11
           1       1.00      1.00      1.00       106
           2       0.99      1.00      0.99       387
           3       1.00      1.00      1.00       479

    accuracy                           0.99       983
   macro avg       1.00      0.89      0.92       983
weighted avg       0.99      0.99      0.99       983
```

In [301…

```
import numpy as np
sample = np.array([[ 4.38,74.09,3.42,26.06,16.53,11.99,0.52,12.72,30.14,0.74,2.2

pred = rf.predict(sample)
print("Raw prediction:", pred)
result = le1.inverse_transform(pred)
print("Final AQI:", result[0])
```

```
Raw prediction: [0]
Final AQI: Bad
```

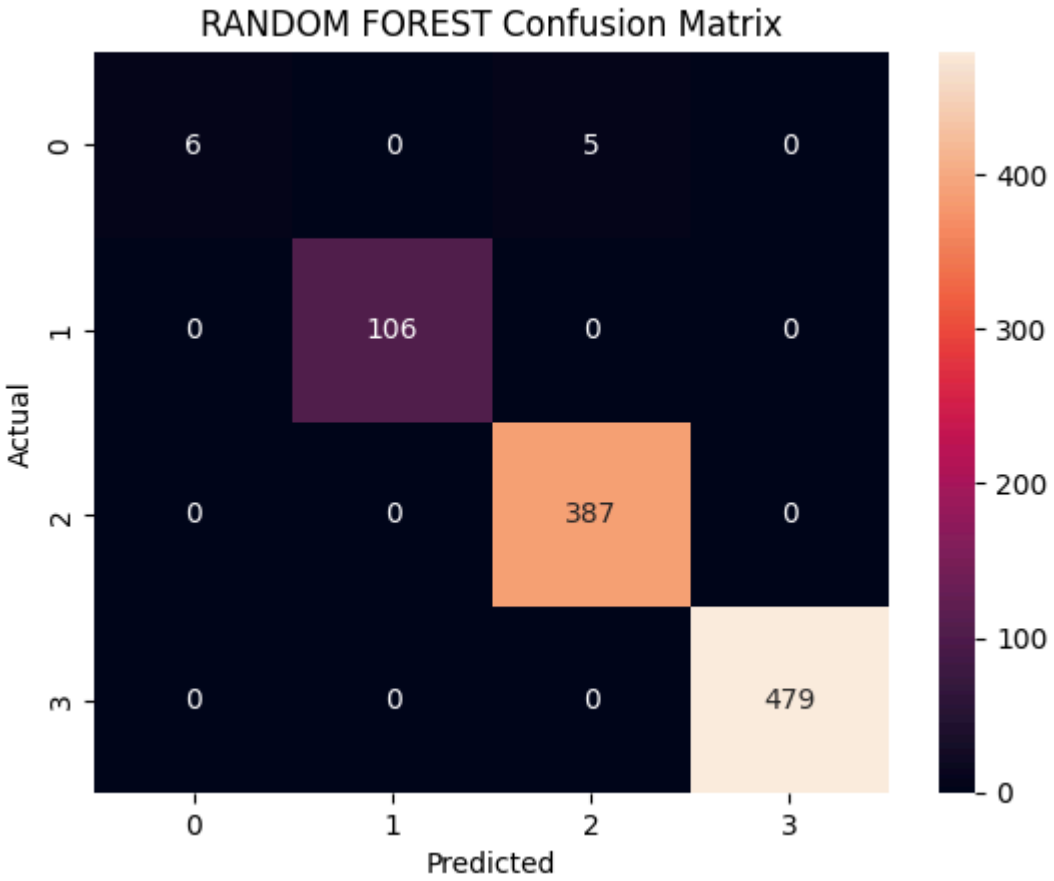
In [302…

```
print(confusion_matrix(y_test,y_pred_rf))
```

```
[[  6   0   5   0]
 [  0 106   0   0]
 [  0   0 387   0]
 [  0   0   0 479]]
```

In [303…

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_rf)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("RANDOM FOREST Confusion Matrix")
plt.show()
```



In [304…

```
from sklearn.svm import SVC
svc = SVC(kernel='rbf')
svc.fit(X_train,y_train)
```

Out[304…

```
▾ SVC
▸ Parameters
```

In [306…

```
y_pred_svc = svc.predict(X_test)
```

In [307…

```
accuracy_score(y_test, y_pred_svc)
```

Out[307…

```
0.9196337741607324
```

In [308…

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
```

```python
svc = SVC()
param_grid = {
    'C': [0.1, 1, 10, 100],          # regularization
    'kernel': ['linear', 'rbf'],     # type of boundary
    'gamma': ['scale', 'auto']       # kernel coefficient
}
grid_svc = GridSearchCV(
    svc,
    param_grid,
    cv=5,
    scoring='f1_weighted',
    n_jobs=-1
)
grid_svc.fit(X_train, y_train)
print("Best Parameters:", grid_svc.best_params_)
print("Best Score:", grid_svc.best_score_)
```

```
Best Parameters: {'C': 100, 'gamma': 'scale', 'kernel': 'linear'}
Best Score: 0.9889585171569127
```

In [309…

```python
from sklearn.svm import SVC
svc1 = SVC(C= 100, gamma ='scale', kernel= 'linear')
svc1.fit(X_train,y_train)
```

Out[309…

```
▾ SVC
▸ Parameters
```

In [310…

```python
import pickle
import pandas as pd
with open("svcfinal.pkl", "wb") as f:
    pickle.dump((svc1, scaler, le1, cols), f)
```

In [312…

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc
print(classification_report(y_test, y_pred_svc))
```

```
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        11
           1       0.91      0.85      0.88       106
           2       0.92      0.94      0.93       387
           3       0.92      0.94      0.93       479

    accuracy                           0.92       983
   macro avg       0.69      0.68      0.68       983
weighted avg       0.91      0.92      0.91       983
```

```
C:\Users\DELL\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2k
fra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\metrics\_classif
ication.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set t
o 0.0 in labels with no predicted samples. Use `zero_division` parameter to contr
ol this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\DELL\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2k
fra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\metrics\_classif
ication.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set t
o 0.0 in labels with no predicted samples. Use `zero_division` parameter to contr
ol this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\DELL\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2k
fra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\metrics\_classif
ication.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set t
o 0.0 in labels with no predicted samples. Use `zero_division` parameter to contr
ol this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

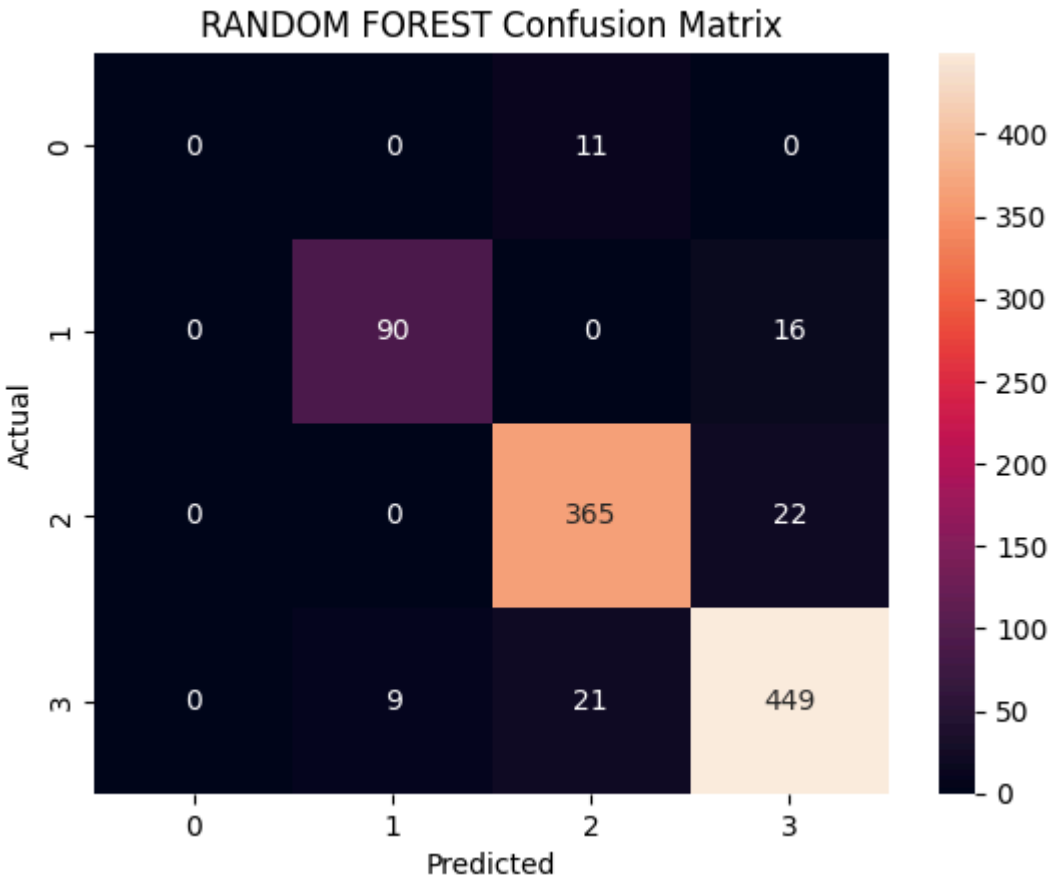
In [313…

```
print(confusion_matrix(y_test,y_pred_rf))
```

```
[[  6   0   5   0]
 [  0 106   0   0]
 [  0   0 387   0]
 [  0   0   0 479]]
```

In [314…

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_svc)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("RANDOM FOREST Confusion Matrix")
plt.show()
```



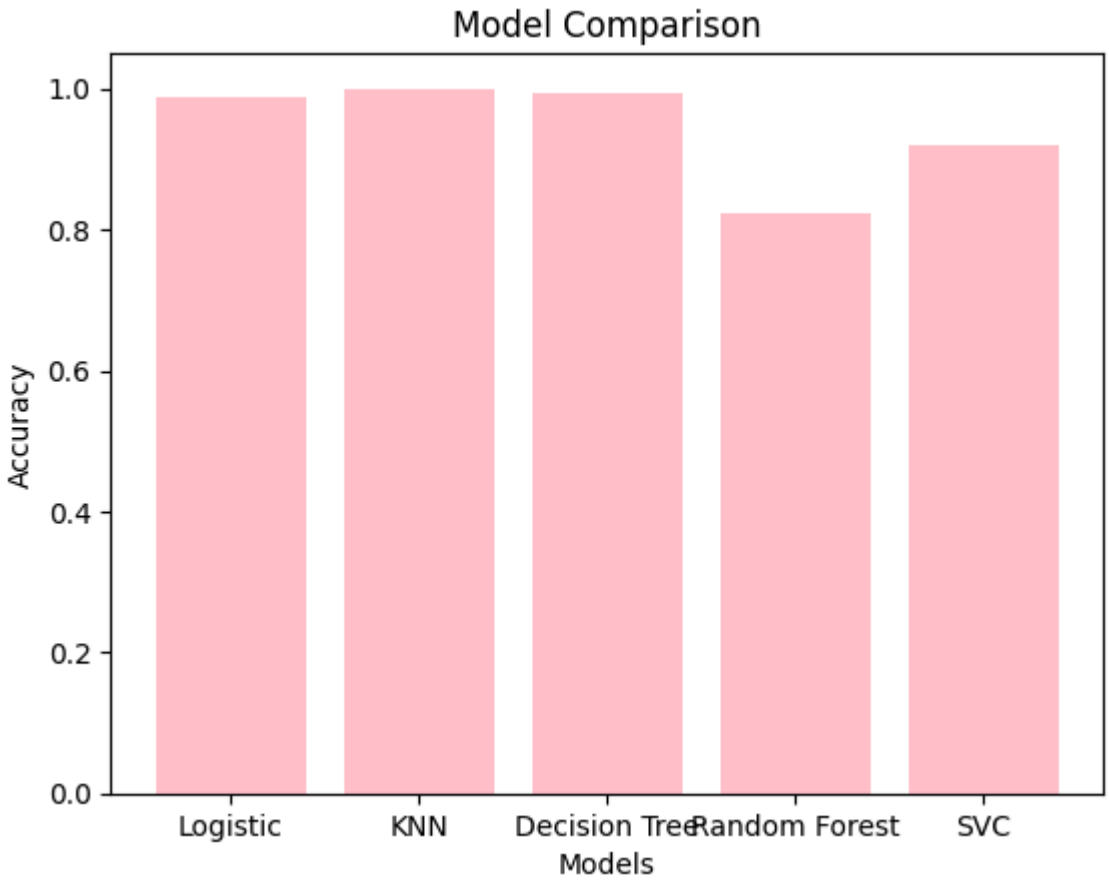
In [316…

```
import numpy as np
sample = np.array([[ 4.38,74.09,3.42,26.06,16.53,11.99,0.52,12.72,30.14,0.74,2.2
pred = svc1.predict(sample)
print("Raw prediction:", pred)
result = le1.inverse_transform(pred)
print("Final AQI:", result[0])
```

```
Raw prediction: [0]
Final AQI: Bad
```

In [317…

```
models = ['Logistic','KNN', 'Decision Tree', 'Random Forest','SVC']
accuracies = [
    accuracy_score(y_test, y_pred),
    accuracy_score(y_test, y_pred_dt),
    accuracy_score(y_test, y_pred_rf),
    accuracy_score(y_test,y_pred_kn),
    accuracy_score(y_test,y_pred_svc)
]
plt.figure()
plt.bar(models, accuracies,color="pink")
plt.title("Model Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.show()
```



In [ ]:
```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
models = {
    "Logistic": LogisticRegression(solver='saga',max_iter=20000),
    "KNN": KNeighborsClassifier(),
    "Decision Tree": DecisionTreeClassifier(max_depth=5),
    "Random Forest": RandomForestClassifier(n_estimators=100, max_depth=10),
    "SVM": SVC()
}
for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=5, scoring='f1_weighted')
    print(name, scores.mean())
```

```
Logistic 0.8046876376594222
KNN 0.9109560158636226
Decision Tree 0.9996810669377554
Random Forest 0.9969425965516777
SVM 0.9541192213663765
```

In [ ]:
```
#In this study, multiple machine learning models Logistic Regression, K-Nearest
# Random Forest, and Support Vector Machine (SVM) were implemented and evaluated
# category (AQI_Bucket) based on various pollutant features. The models were ass
# with the weighted F1-score to ensure reliable and unbiased performance evaluat

#The results showed that Decision Tree and Random Forest achieved perfect F1-sco
#  However, this near-perfect performance is attributed to the inherent nature o
#  where the target variable AQI_Bucket is derived from pollutant concentrations
#  predefined threshold ranges. This makes the relationship between input featur
```

```
#  highly deterministic and easily separable. As a result, tree-based models are
#  decision rules, leading to overfitting and reduced generalization capability

#On the other hand, Logistic Regression achieved a high F1-score of 0.80, demons
#  strong predictive performance while maintaining better generalization. Its si
#  interpretability, and lower tendency to overfit make it a more reliable choic
#  The SVM model also performed well with an F1-score of 0.954, but it is comput
#  and less interpretable. KNN showed comparatively lower performance due to its
#  distribution and feature scaling.

#Based on these observations, Logistic Regression or Random Forest is selected a
#  for this task. It provides an optimal balance between accuracy, robustness, a
#  making it suitable for practical applications. Overall, this study highlights
#  not relying solely on accuracy metrics, but also considering model behavior,
# and generalization ability when selecting the best model.
```

In [ ]:
```
print(le1.classes_)
```

```
['Bad' 'Good' 'Moderate' 'Satisfactory']
```